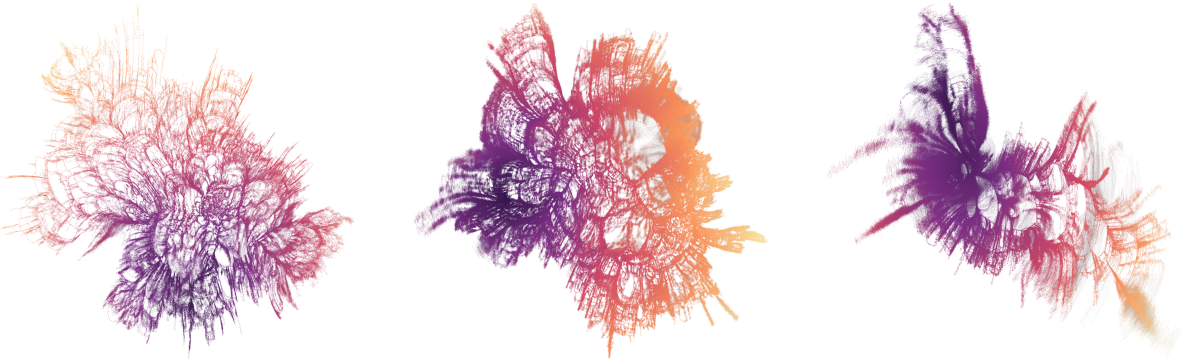
# Round-Robin SGD: Accelerating Stress Minimization for Graph Drawing on GPUs

Kohsei Mochizuki Yosuke Onoue*

Nihon University



(a) `belgium_osm` $|V| = 1,441,295$, $|E| = 1,549,970$

(b) `germany_osm` $|V| = 11,548,845$, $|E| = 12,369,181$

(c) `italy_osm` $|V| = 6,686,493$, $|E| = 7,013,978$

Figure 1: Road-network layouts generated by RoundRobin Sparse SGD: (a) Belgium, (b) Germany, and (c) Italy. The largest example contains 11,548,845 vertices and 12,369,181 undirected edges. Colors indicate normalized graph-hop distance from a fixed source; positions are computed layouts rather than geographic coordinates. The teaser uses 15 iterations and seed 0, with 200, 40, and 50 pivots for Belgium, Germany, and Italy. Fewer pivots limit the size of the distance table on the two larger networks. subsection 4.3 evaluates Italy and Germany at a common 40 pivots.

## Abstract

Parallelizing vertex-pair updates in stochastic gradient descent (SGD) offers a way to accelerate stress minimization for graph drawing, but updates sharing a vertex can conflict. Atomic locks prevent conflicting accesses, yet each pair update still reads and writes endpoint coordinates in global memory. Inspired by the batching idea in mini-batch SGD, we propose round-robin GPU schedules that group independent updates and reuse coordinates while preserving the vertex-pair update rule. RoundRobin Full SGD interleaves 16 segments of the inner round sequence across outer rounds, reusing coordinates within each tile segment. RoundRobin Sparse SGD groups one-sided updates by destination and two-sided updates by disjoint endpoints. We compare both schedules with sequential CPU and lock-based GPU implementations over 15 iterations from matched initial layouts. RoundRobin Full SGD shortens iterations on 17 of 19 comparable graphs, with a median CPU-relative speedup of 9.98×. Its median final stress increase over CPU is 0.04%, and the largest graph-median increase is 20.8%. RoundRobin Sparse SGD shortens iterations on all 37 comparable graphs, with a median CPU-relative speedup of 44.49×, or 3.77× including preprocessing and setup. Its median final stress increase over CPU is 0.02%. With 40 pivots, RoundRobin Sparse SGD lays out road networks of 6.69 million and 11.55 million vertices in 43.45 s and 80.32 s, respectively, including preprocessing.

**Keywords:** Graph drawing, stress minimization, stochastic gradient descent, GPU parallelization, round-robin scheduling

**Index Terms:** Human-centered computing—Visualization—Visualization techniques—Graph drawings

*e-mail: onoue.yousuke@nihon-u.ac.jp

## 1 Introduction

Stochastic gradient descent (SGD) is a promising approach to efficient stress minimization for graph drawing. Placing vertices so that network structure can be understood is a central task in graph drawing; stress minimization seeks to reproduce graph distances in the drawing space [20, 31]. Zheng et al. [49] proposed SGD that updates one vertex pair at a time and reported fast, stable progress toward lower stress than stress majorization in their experiments. A replication study, however, found that the advantage in final stress does not always hold [3].

SGD applies both to layouts that consider all vertex pairs and to large graphs for which the update set is reduced. Let $X$ denote vertex coordinates, $d_{ij}$ the shortest-path distance between vertices, and $w_{ij} = d_{ij}^{-2}$ the weight. Stress is defined as

$$S(X) = \sum_{i<j} w_{ij}\left(\|x_i - x_j\| - d_{ij}\right)^2. \tag{1}$$

We refer to the scheme that updates all vertex pairs as *Full SGD*. *Sparse SGD* optimizes Sparse Stress by SGD, approximating the objective with actual edges and terms involving a small set of representative vertices, or pivots, to handle larger graphs [36, 49].

Even after reducing the update set, a sequential implementation leaves room for parallel execution. Full SGD processes $n(n-1)/2$ pairs per iteration, and Sparse SGD also performs many updates. Our CPU baselines execute these updates sequentially. Processing multiple terms simultaneously could reduce runtime, but pairs that share a vertex can conflict when reading and writing coordinates. Locks implemented with atomic operations offer one solution, at the cost of acquisition retries and coordinate accesses.

We propose GPU methods based on round-robin (RR) scheduling to accelerate Full and Sparse SGD by organizing vertex-pair updates according to their dependencies. RR scheduling partitions

all pairings into rounds in which each participant appears once. For Full SGD, we construct RR schedules at two levels, between vertex blocks and within tiles, combining parallel independent updates with coordinate reuse. We interleave segments of inner rounds across the outer schedule to limit consecutive updates against one partner block. For Sparse SGD, RR rounds for pivot pairs are extended with disjoint actual-edge updates. We alternate segments of this two-sided schedule with one-sided updates grouped by their non-pivot destination; pivots remain read-only during each one-sided segment. Conflict-free scheduling is already used in parallel SGD [30, 37, 51]; our focus is on execution schemes tailored to these two update structures in graph drawing.

Shorter iteration times do not necessarily imply less time to reach the same layout quality. Changes in update order induced by RR and differences in numerical precision can affect the final layout. We therefore compare the schemes with their CPU and Atomic counterparts in terms of iteration time, stress after a fixed number of iterations, and attainment of a quality target. We implement the schemes using wgpu and WGSL.

After 15 iterations, RoundRobin Full SGD shortens iterations on 17 of 19 comparable graphs and keeps the median stress ratio at most 1.05 on 18; 16 meet both criteria. RoundRobin Sparse SGD meets both criteria on 36 of 37 comparable graphs. We also measure time to a common stress target and compare split and unsplit Full schedules on preferential-attachment graphs of increasing size. A separate road-network experiment computes coordinates for more than eleven million vertices with a reduced pivot set.

## 2 Related Work

### 2.1 Stress and SGD-Based Graph Drawing

Force-based layout represents graph structure through geometric interactions, from Eades' spring embedder [15] and Fruchterman–Reingold [17] to ForceAtlas2 [29]. Kamada–Kawai instead derives ideal distances from graph shortest paths [31]. Distance fidelity also connects graph drawing to classical MDS [22, 43], nonmetric scaling [34], and nonlinear distance-preserving mapping [42]. Stress majorization [9, 20] iteratively optimizes an auxiliary function for the distance objective.

Stochastic approximation [40] underlies a broad family of iterative optimization methods. For graph drawing, Zheng et al. [49] give simple vertex-pair updates and a learning-rate schedule for stress minimization. Börsig et al. [3] confirm their practical robustness while showing that final stress need not always improve over majorization. We retain these updates and evaluate their parallel execution.

Other layout methods change the objective or impose additional constraints. Simulated annealing combines drawing criteria [7]; constrained layouts include separation constraints, distance constraints, and overlap removal [12–14]. More recent frameworks include multicriteria $(\mathrm{SGD})^2$ [1] and UNICON [48]. Our schedules change how existing updates execute, rather than adding objectives or constraints.

### 2.2 Scalable Graph Layout

Multilevel methods accelerate layout by constructing and refining coarse graph representations [18, 26, 45]. Hierarchical force approximation [2] and potential-field methods [25] reduce the cost of distant interactions; Hu combines multilevel processing with force approximation [28]. These methods provide alternatives to explicitly processing all vertex pairs.

Distance-based approaches include high-dimensional embeddings [27], PivotMDS [6], low-rank stress majorization [32], and Maxent-Stress [19]. Sparse Stress reduces the objective to edges and region-weighted pivot terms [36]; Zheng et al. [49] optimize this approximation with SGD. Our sparse scheme uses the same construction, separating term reduction from parallel execution of the retained terms.

Dimensionality reduction offers another route to scale: tsNET uses t-SNE for graph layout [33, 44], and DRGraph combines sparse distances, negative sampling, and multilevel processing [50]. CoRe-GD learns stress-oriented layouts with a graph neural network and a coarsening hierarchy [23]. We directly execute existing stress updates without a learned model. Public graph repositories [8, 41] and recent evaluation guidance [11] support reproducible assessment across graph families, runtime measures, and quality criteria.

### 2.3 GPU Execution and Update Conflicts

GPU layout includes multilevel graph drawing [16], GPU-friendly t-SNE optimization [38], and pangenome layout optimized for irregular memory access [35]. GPU stress majorization maps a constrained layout problem to a conjugate-gradient solver [46]. Pairwise SGD exposes a different unit of parallel work: one update reads and can move both endpoints. In SGD, learning rates and sample order affect optimization [4, 5]; mini-batches expose work that can be computed in parallel [10]. Graph drawing additionally requires coordination when pair updates modify shared coordinates.

Hogwild! tolerates asynchronous conflicts under sparsity assumptions [39], whereas CYCLADES constructs independent updates [37]. Parallel matrix factorization also emphasizes locality and load balancing [51]. Kaleem et al. [30] compare GPU locks and matching and show that a conflict-free schedule is not always faster. We therefore combine independence with tile-level coordinate reuse.

Wang et al. [47] directly address conflict-free GPU SGD for graph drawing, assigning pairs with a common update destination to one thread and extending the method to Sparse Stress. Our one-sided Sparse phase also groups work by destination: each thread writes one non-pivot coordinate and all pivots are read-only until the phase ends. For an edge or a pivot pair, however, our update moves both endpoints using their pre-update coordinates. We assign such pairs to disjoint matchings and synchronize between rounds. For Full SGD we retain both blocks in shared memory across successive matchings. Our measurements compare these choices with sequential SGD and a lock-based GPU implementation. Their performance relative to Wang et al.'s schedule remains open. Round-robin pair construction is distinct from random reshuffling; convergence guarantees for the latter [24] do not directly establish behavior for nonconvex stress under our schedule.

## 3 Round-Robin GPU Scheduling for Stress Minimization

We present RoundRobin Full SGD and RoundRobin Sparse SGD to accelerate Full and Sparse SGD on the GPU.

### 3.1 SGD and Atomic Updates

We describe the SGD updates for a connected, undirected, unweighted graph $G = (V, E)$ with $n$ vertices and $m$ edges. Let $x_i$ be the two-dimensional coordinate of vertex $i$, and $d_{ij}$ the shortest-path distance. Zheng et al.'s SGD [49] updates vertex pairs sequentially; we call the formulation over all vertex pairs Full SGD. An *iteration* is one complete pass over the update set, with distance computation and term construction treated as preprocessing.

Following Zheng et al. [49], the displacement is computed from the difference between the current and target distances. For $x_i \neq x_j$, the pre-update coordinates give

$$r_{ij} = \frac{\|x_i - x_j\| - d_{ij}}{2} \frac{x_i - x_j}{\|x_i - x_j\|}. \tag{2}$$

With endpoint weights $w_{ij}^{(i)}, w_{ij}^{(j)}$ and learning rate $\eta_t$ at iteration $t$, the update is

$$\begin{aligned} \mu_i &= \min\{1, \eta_t w_{ij}^{(i)}\}, \quad x_i \leftarrow x_i - \mu_i r_{ij}, \\ \mu_j &= \min\{1, \eta_t w_{ij}^{(j)}\}, \quad x_j \leftarrow x_j + \mu_j r_{ij}. \end{aligned} \tag{3}$$

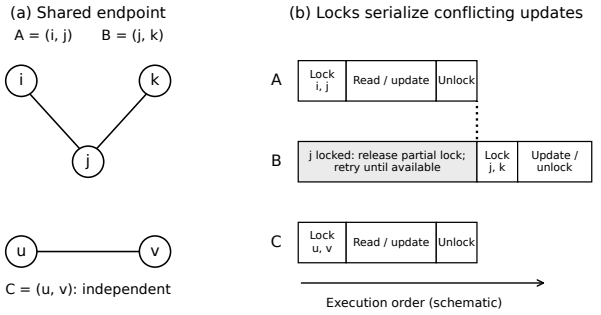



Figure 2: The Atomic scheme holds both endpoint locks while reading and updating coordinates, then releases them. Updates sharing a vertex are ordered by lock acquisition. Gray intervals indicate retries; their lengths do not represent measured waiting times.

Full SGD uses $d_{ij}^{-2}$ at both endpoints and updates both from the same pre-update $r_{ij}$. Each implementation applies its own numerical handling when the current distance is extremely small.

Sparse SGD optimizes Sparse Stress using actual edges and terms representing distant relationships through a small pivot set [36, 49]. Let $P$ be the pivot set and $p = |P|$. Each vertex is assigned to the region of its nearest pivot, and weights are derived from region information. Compared with Full SGD, the sparse formulation reduces the number of terms to $O(m+np)$ and avoids storing all-pairs distances. It uses endpoint-specific weights in (3) and fixes an endpoint whose weight is zero. A non-edge pair between a non-pivot and a pivot moves only the non-pivot endpoint; actual edges and pivot–pivot pairs move both endpoints. We call these one-sided and two-sided updates, respectively. A two-sided update uses the same pre-update $r_{ij}$ for both endpoints, and actual-edge terms take precedence when removing duplicates.

The CPU implementation shuffles the update set at every iteration and uses coordinates reflecting the immediately preceding update. If $(j,k)$ follows $(i,j)$, it reads the updated $x_j$. Executing both directly in parallel can cause write conflicts or inconsistent reads of the two endpoints. Making coordinate additions atomic alone does not make the entire pair update mutually exclusive.

We implement the Atomic baselines directly using the Metal API. The Atomic scheme locks both endpoints and executes the read–update–write sequence exclusively [30]. It attempts to acquire the lower-ID endpoint first, applies (2) and (3) after obtaining both locks, and releases them after writing. If only one lock is acquired, it releases that lock and retries. Atomic Full uses device-scope fences after successful lock acquisition and before release, and atomic operations on coordinate components. In Atomic Sparse, relaxed atomic lock operations alone do not guarantee the visibility of coordinate updates to the next lock holder. We therefore place device-scope memory fences after acquiring both endpoint locks and before releasing them, ensuring that coordinate updates made by the previous lock holder are visible to the next holder when it reads the coordinates. In Figure 2, updates A and B share vertex $j$ and proceed in sequence, while independent update C can run concurrently.

The Full and Sparse Atomic implementations differ in completion handling and update ordering. Full retries lock acquisition and checks the update count after GPU execution. Sparse gathers unfinished terms on the CPU and resubmits them. Even one-sided updates lock both endpoints to prevent another update from changing a pivot while it is being read. Atomic Full submits pairs without an explicit shuffle, whereas Atomic Sparse shuffles pair IDs before each iteration. In both implementations, when two updates share a vertex, the thread that first acquires both endpoint locks applies its update first. Even with a fixed submission order, the relative timing of lock attempts and retries can change which update runs first. This runtime ordering does not guarantee a uniformly random permutation of the pairs.

### 3.2 Mini-Batch SGD

Mini-batch SGD groups several objective terms into a batch and updates the parameters using their gradients [5]. Compared with processing one term at a time, computation within a batch exposes parallel work that can be assigned to a GPU. For an objective expressed as a sum or average of terms $f_a(\theta)$, a conventional update for a selected set $B_t$ is

$$\theta_{t+1} = \theta_t - \frac{\alpha_t}{|B_t|} \sum_{a \in B_t} \nabla f_a(\theta_t), \tag{4}$$

where $\alpha_t$ is the learning rate. All gradients in a batch are evaluated at the same $\theta_t$, then aggregated for a single parameter update.

This idea of processing multiple terms together motivates our grouping of vertex-pair updates for graph drawing. Directly executing updates that share a vertex can cause conflicting coordinate reads and writes. We therefore construct groups of updates that can run concurrently, applying (2) and (3) to each pair. The resulting execution groups retain the existing pairwise update rule rather than introducing the gradient average in (4).

### 3.3 RoundRobin Full SGD

Pairs with disjoint endpoints can be updated independently without aggregating gradients. An update of $(i,j)$ reads and modifies only $x_i$ and $x_j$, so it does not change the inputs of a disjoint pair $(u,v)$. A set of such pairs is a *matching*. Applying the same update rule to pairs within one matching introduces no mathematical dependence between their sequential and simultaneous execution, allowing parallelization without shared-vertex locks.

RR partitions all pairings into rounds in which each participant appears once. For four vertices, $(1,2),(3,4)$; $(1,3),(2,4)$; and $(1,4),(2,3)$ cover all six pairs exactly once in three rounds. For an even number of participants, the number of rounds is one less than the number of participants. For an odd number, we add a dummy participant and discard pairs containing it. Pairs within a round run in parallel, and the next round waits for completion.

Independence within a round does not imply that different round orders yield the same final layout. RR imposes ordering constraints that differ from the CPU's unrestricted shuffle of all pairs, so we evaluate both iteration time and layout quality.

RoundRobin Full SGD constructs independent updates at two levels: between blocks and within blocks. We group vertices in ID order into blocks of at most $T$ vertices, where $T$ is the *tile size*. All pairs between two distinct blocks form a tile; pairs within one block form a self-tile. We first choose tiles that can run concurrently and then choose concurrent pair updates inside each tile.

The outer RR schedule ensures that each block participates in at most one tile in a round. The upper triangle in Figure 3(a) contains tiles between distinct blocks. Tiles with the same R label share no blocks and can run in parallel. Diagonal self-tiles S are packed into additional rounds with disjoint blocks, and duplicated lower-triangular tiles are omitted. This covers every vertex pair exactly once.

Within a tile, we construct matchings in which each vertex has only one partner. Numbering vertices in each of two distinct blocks from 0 to $T-1$, inner round $r$ pairs vertex $a$ with vertex $(a+r)$ mod $T$ in the other block. Rounds $r = 0,\ldots,T-1$ cover all $T^2$ pairs once. Each number in Figure 3(c) appears once in every row and column. Since rows and columns represent different blocks, diagonal entries are valid pairs. Self-tiles use RR on the actual vertices, excluding self-pairs and duplicates.

At every iteration, we shuffle the outer round order, including self-tile rounds, and the inner matching order for each outer round.

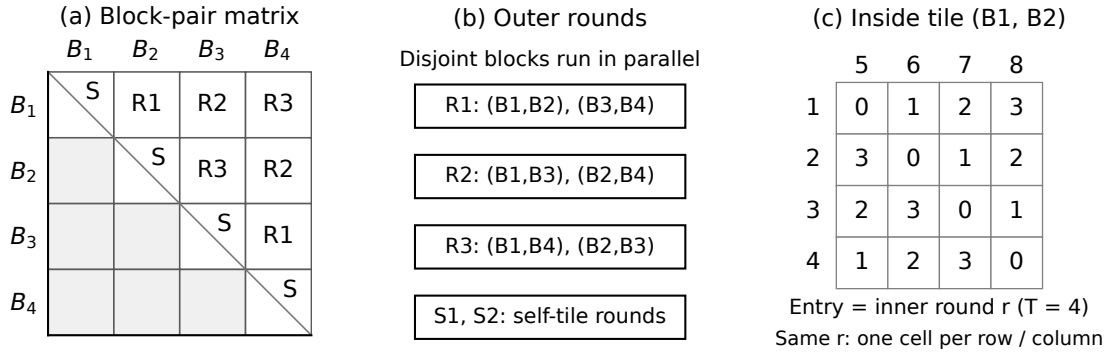



Figure 3: Two-level scheduling in RoundRobin Full SGD. (a) Partition all vertex pairs into blocks. (b) Process tiles with disjoint blocks in the same outer round. (c) Partition a tile between distinct blocks into cyclic matchings. Here $T = 4$; row vertices 1–4 and column vertices 5–8 are distinct sets. Self-tiles use a separate RR schedule. The illustrated round order is explanatory; execution orders are shuffled. Each inner sequence is split into segments, and all outer rounds are visited before advancing to the next segment.

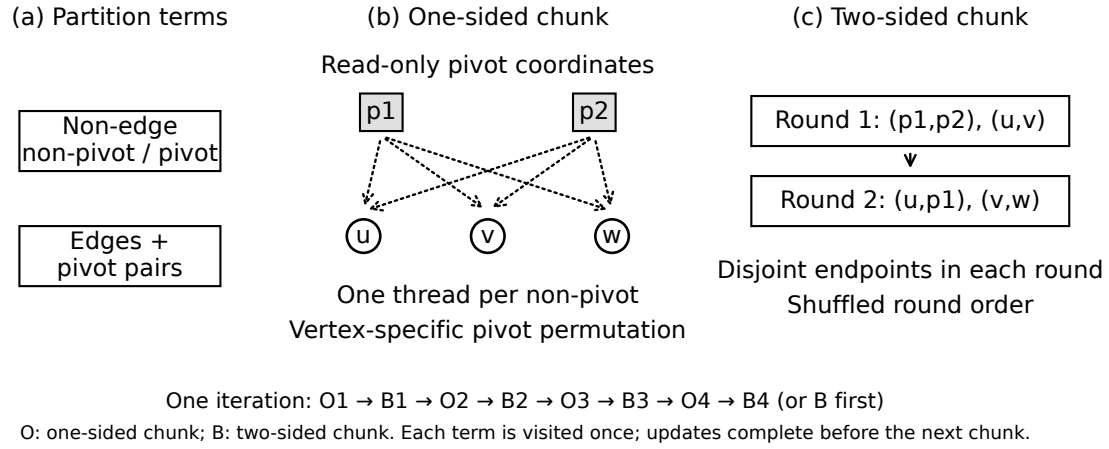



Figure 4: Update separation in RoundRobin Sparse SGD. (a) Partition the update set. (b) In the one-sided phase, each thread owns a non-pivot vertex and updates its local coordinate while reading pivots without modifying them. Dotted arrows denote coordinate reads. (c) In the two-sided phase, process successive matchings with disjoint endpoints. Each phase is split into four segments and the segments alternate, covering all terms once per iteration.

Tiles of the same type and size within an outer round share the inner order. Completing an entire tile before visiting another outer round can concentrate successive updates on the same partner block. We therefore split each shuffled inner sequence into $C = \min(16, T)$ contiguous segments and visit every outer round for segment 0, then every outer round for segment 1, and so on. For a sequence of length $L$, segment $c$ contains positions $\lfloor cL/C \rfloor$ through $\lfloor (c+1)L/C \rfloor - 1$. The split does not sample or discard updates: every pair is still processed once per iteration with the same learning rate. It changes the interleaving of different outer rounds. We selected 16 segments in a preliminary investigation on `wiki-Vote`; the evaluation keeps this count fixed across graphs.

Each tile segment is assigned to one workgroup, a group of threads that can share memory and synchronize with barriers. The workgroup loads coordinates from the participating blocks and processes each inner matching in parallel, retaining coordinates in shared memory throughout that segment. All threads participate in a barrier after each matching so that the next matching uses updated coordinates. At the end of the segment, coordinates are written back to global memory before another outer round uses them. Blocks smaller than $T$ are padded with dummy vertices; only pairs with two actual endpoints are updated. Dispatches execute in queue order; the CPU waits for GPU completion at the end of each iteration.

Coordinate reuse reduces global-memory reads and writes within each segment. For two blocks of $T$ vertices, a tile contains $T^2$ pairs. Reading and writing both endpoints for each pair requires $2T^2$ coordinate reads and $2T^2$ writes. Loading and storing $2T$ coordinates in each of $C$ segments requires $2CT$ reads and $2CT$ writes. These counts treat one two-dimensional coordinate as one access. At $T = 1024$ and $C = 16$, the coordinate-access count is reduced by a factor of 64 relative to per-pair access, while dispatch and transfer counts increase relative to an unsplit tile. Distance-matrix accesses remain, so these counts do not imply an equal runtime speedup.

Tile size affects both reuse and parallelism. Larger $T$ reduces the number of outer rounds, but increases updates and synchronization within each tile and reduces the number of tiles that can run concurrently. It also changes update order and can affect quality. All-pairs

updates and distance storage remain $O(n^2)$; the scheme improves work grouping and coordinate accesses.

### 3.4 RoundRobin Sparse SGD

RoundRobin Sparse SGD parallelizes the one-sided and two-sided updates of the existing Sparse SGD formulation described in Section 3.1. CPU, Atomic, and RoundRobin Sparse SGD use distances and endpoint weights from common preprocessing.

One-sided updates are grouped by the vertex they modify and processed independently. In Figure 4(b), one thread is assigned to each non-pivot vertex. It loads the coordinate into a local variable, sequentially applies the pivot terms assigned to that segment, and writes back at the end of the segment. Terms that modify pivots run in a separate stage. Our implementation excludes pivot vertices from one-sided update targets and performs no pivot writes in this phase. This allows threads to read shared pivots while reusing their assigned vertex coordinates.

Two-sided updates avoid concurrent writes through RR for pivot pairs and matchings for actual edges. After constructing RR rounds for pivot pairs, we reorder the remaining edges and insert each into the first round in which neither endpoint is used, creating a round if necessary. A pivot pair that is also an actual edge appears only once. Every round is a matching and can run without locks, although edges concentrated at particular vertices can require many rounds.

We call the stages for the two update types the one-sided and two-sided phases and wait for a stage to finish before proceeding. At each iteration, we shuffle a common pivot permutation and the two-sided round order. A permutation based on vertex ID, iteration, and seed determines each vertex's pivot order. We alternate four segments of each phase, selecting the starting phase per iteration and covering every term once. The update rule, weights, learning rate, and displacement cap are unchanged; execution order differs from the CPU term shuffle.

## 4 Evaluation

We evaluate whether faster vertex-pair updates reduce the time needed to obtain a layout of comparable stress [11]. The evaluation covers runtime and final stress relative to sequential CPU and lock-based GPU implementations after 15 iterations, elapsed SGD time to a shared stress target, a comparison of split and unsplit schedules on preferential-attachment graphs, and preprocessing-inclusive runtime for road networks with millions of vertices.

We ran each implementation separately on an Apple M4 Pro with 24 GiB unified memory and macOS 15.7.2. CPU is sequential Rust with f64 arithmetic; Atomic uses native Metal, and both RoundRobin methods use Rust, wgpu, and WGSL [21] with the Metal backend and f32 updates. The implementations differ in numerical precision and execution paths, so the measured speedups reflect these differences as well as scheduling. Within each family, inputs, seeds, initial layouts, learning rates, and Sparse preprocessing are matched. We remeasured RoundRobin Full SGD and retained the archived CPU, Atomic, and Sparse measurements from the same machine. These main comparisons combine different measurement dates and do not isolate transient system load. The additional preferential-attachment experiment measures CPU and both Full GPU schedules during the same session. We use 15 iterations with Zheng et al.'s schedule [49], $\varepsilon = 0.1$, five seeds for runtime comparisons, and ten for convergence diagnostics. We fix Full at $T = 1024$ with 16 segments and retain four alternating segments per phase in Sparse. Tile-size sensitivity is outside this evaluation. We fix Sparse at 200 pivots in the CPU and GPU comparisons so that every method optimizes the same approximation. RoundRobin Full SGD and RoundRobin Sparse SGD workgroups contain 1024 and 256 threads, respectively.

We evaluated Full SGD on 20 graphs and Sparse SGD on 38, using datasets based on the SuiteSparse collection [8]. For `com-Amazon`, `com-DBLP`, and `loc-Gowalla`, all compared methods use the same official SNAP inputs. We remove directions, weights, numerical zeros, self-loops, and duplicate edges, and verify identical preprocessed largest connected components by SHA hashes. The comparisons use a 120-second process threshold and an observed 8-GiB RSS threshold, sampled approximately every 200 ms. A stopped preliminary run prevents formal runs for that setting; failed, unlaunched, and unavailable runs are excluded from paired ratios, rather than assigned artificial values. RSS monitoring is not a strict instantaneous or dedicated-GPU-memory limit.

SGD time includes shuffling, dispatch, completion waits, and resubmission, but excludes preprocessing, initialization, stress evaluation, and coordinate saving. For runtime comparisons we divide the 15-iteration total by 15; the road-network experiment reports the total. Algorithm time additionally includes preprocessing, GPU setup, transfers, and postprocessing; process time also includes input, evaluation, and output. Quality comparisons use the full stress objective in (1), computed exactly up to 8,000 vertices and otherwise estimated from 256 fixed sources independent of the pivots. For jointly successful seeds, speedup is CPU time divided by GPU time, and stress ratio is GPU stress divided by CPU stress. We report graph-level medians, with seed minima and maxima rather than confidence intervals.

### 4.1 Comparison with CPU and Atomic

Using $T = 1024$ for Full and $p = 200$ for Sparse, Figure 5 compares mean iteration times for each graph. We also report final stress ratios to assess layout quality.

RoundRobin Full SGD is faster on 17 of 19 comparable graphs, with median speedups of 0.855–23.469×. The two slowdowns occur on `dwt_1005` and `1138_bus`. For `fe_4elt2`, speedup is 23.469× with a stress ratio of 1.010; for `wiki-Vote`, the values are 16.928× and 1.026. Eighteen graphs have a median stress ratio at most 1.05, and sixteen combine this quality criterion with faster iterations. The remaining quality gap is on `bcsstk17`, whose ratio is 1.208. Atomic is faster on 13 of 19 graphs, with speedups of 0.064–3.365× and stress ratios of 1.001–2.921. Its recording of pair-update order and pre-update coordinates contributes work beyond locking, so the difference between RoundRobin and Atomic is not a measurement of lock overhead alone.

On both `fe_4elt2` and `wiki-Vote`, all ten RoundRobin traces reach the target (Figure 6). Their median attainment times are 0.463 and 0.385 s, compared with CPU times of 11.593 and 6.691 s.

RoundRobin Sparse SGD is faster on all 37 comparable graphs, with speedups of 6.136–55.053× and stress ratios of 0.987–1.067. Thirty-six graphs combine faster execution with a stress ratio at most 1.05. Atomic is faster on two of 37 graphs: its speedups are 0.869–1.042× and stress ratios 0.971–1.139. Only one graph combines Atomic acceleration with a stress ratio at most 1.05.

Of 100 planned Full runs per method, CPU completed 95 (five memory stops), Atomic completed 95 (five withheld after preliminary failure), and RoundRobin Full SGD completed all 100. Of 190 Sparse runs per method, CPU completed 181 and Atomic 180. Both withheld five after preliminary failure and had three memory stops. CPU had one additional timeout and Atomic two. RoundRobin Sparse SGD completed all 190. The pairwise ratios above use every seed completed by CPU and the GPU method under comparison. For the three-way plot in Figure 5, we require all three methods to complete the same seed.

### 4.2 Convergence toward a Quality Target

The Sparse convergence runs retain the 200 pivots from the timing comparison. Changing the pivot set here would change the objective as well as the trajectory toward the target. We save layouts at iterations 0, 1, 2, 3, 5, 8, 10, and 15 while retaining the same

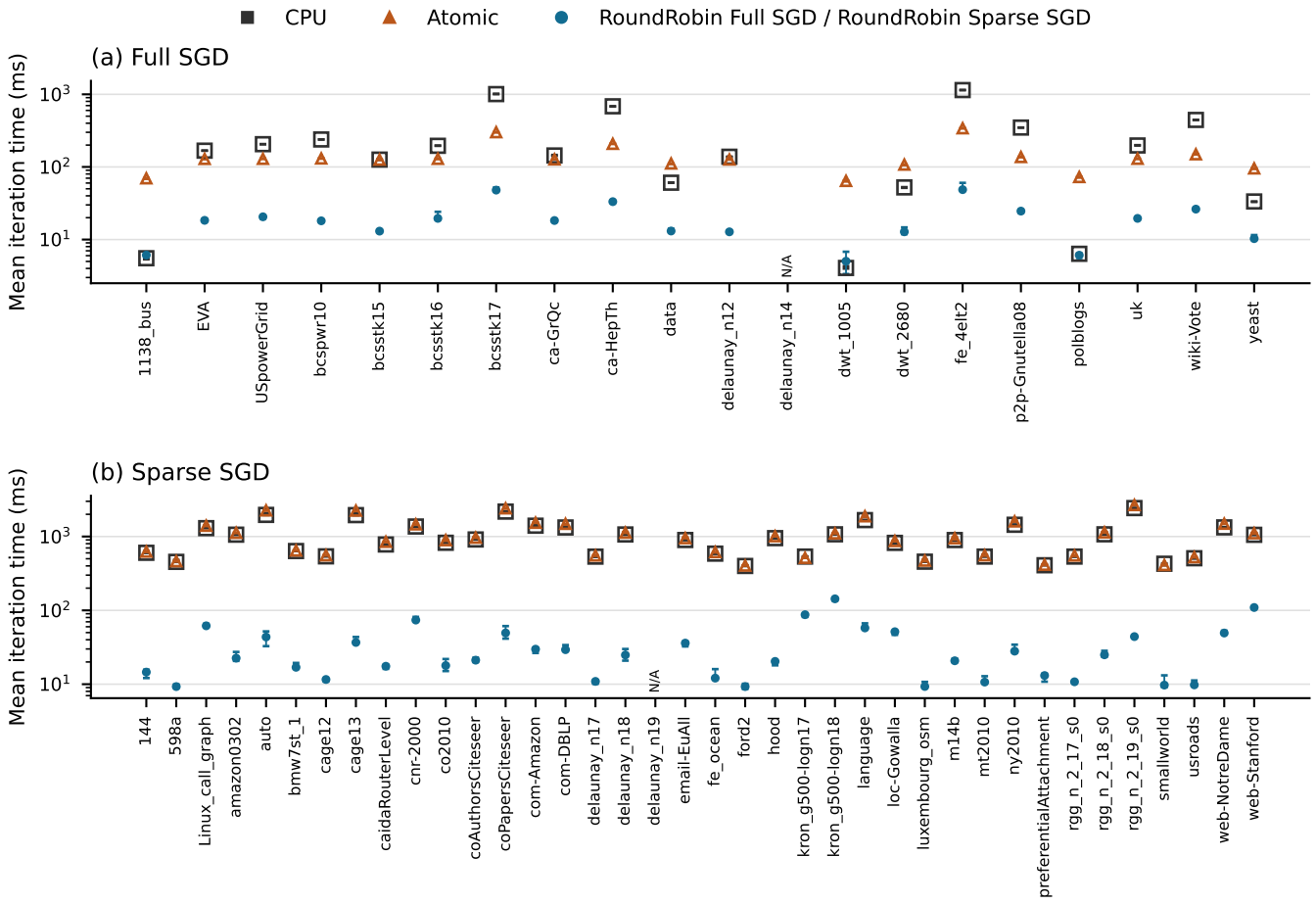



Figure 5: Mean time per iteration for sequential CPU, Atomic GPU, and RoundRobin GPU SGD. For each seed we divide the SGD time by 15. Symbols show the median and bars the range over seeds completed by all three methods (95 Full and 179 Sparse triples). N/A marks graphs without a complete triple. The vertical axis is logarithmic.

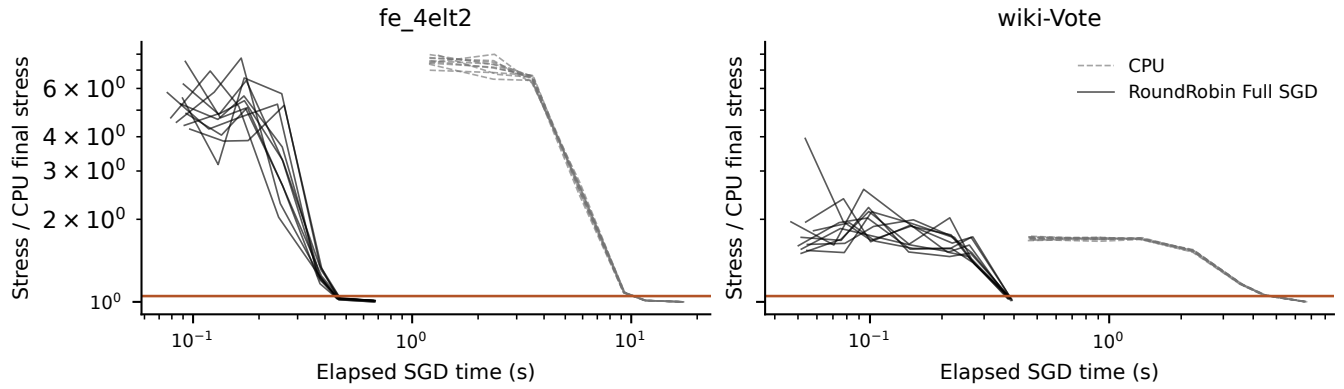



Figure 6: Full SGD diagnostic runs, ten seeds per method. Each trace joins saved checkpoints from iteration 1 to 15 for one seed. Stress is relative to that seed's final CPU stress. The horizontal line marks the 1.05 target.

learning-rate schedule. The target is at most 1.05 times the same-seed CPU final stress. We apply the same threshold to the CPU checkpoints. Table 1 reports the first saved checkpoint at or below the target for each seed. Times exclude saving and intermediate readback.

Across the six Full graphs, 55 of 60 runs attain the target and 55 remain within it at iteration 15. The five Full misses occur on `dwt_2680` (four seeds) and EVA (one seed), so segmentation does not guarantee the target for every initialization. All 30 Sparse runs attain it and remain within it at iteration 15. The recorded checkpoints bound the first observed attainment; they do not identify the exact crossing between saved iterations.

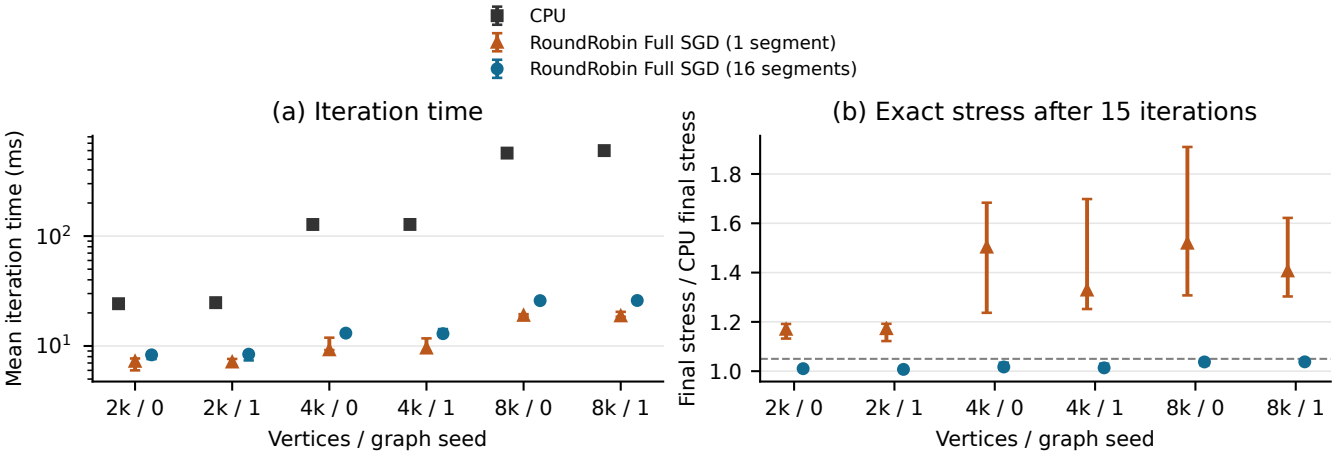



Figure 7: Full SGD on preferential-attachment graphs. Each vertex count has two graph seeds; points are medians and bars are minima and maxima over five matched initial layouts. (a) Mean iteration time on a logarithmic axis. (b) Exact final stress relative to same-seed CPU stress. The horizontal line marks 1.05. Both GPU variants use the same update rule, tile size, and per-iteration completion wait.

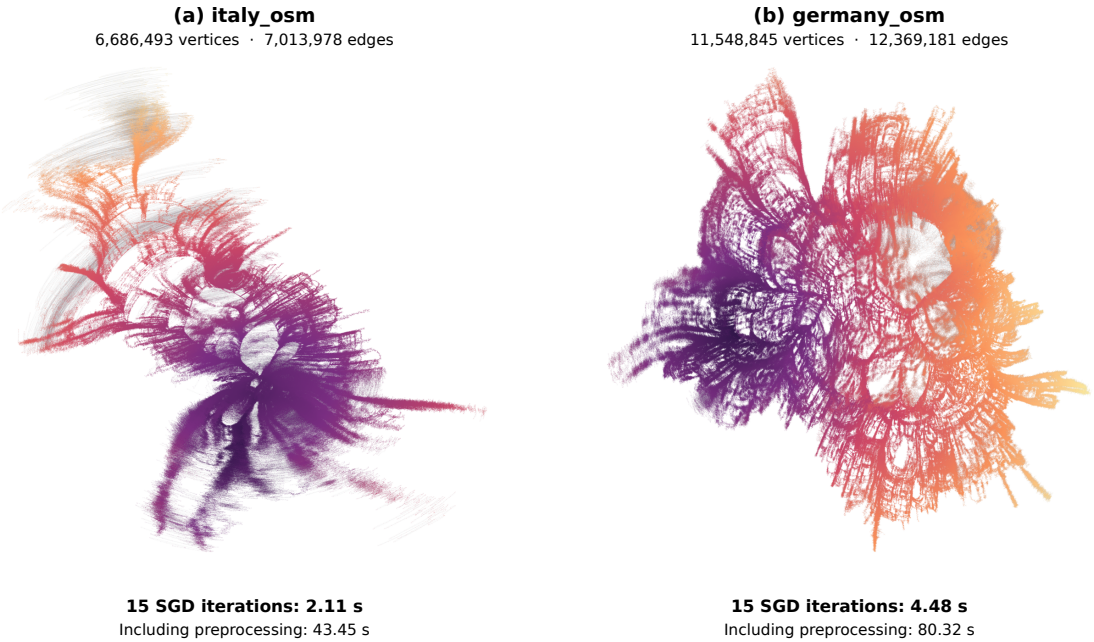



Figure 8: RoundRobin Sparse SGD layouts of Italy and Germany with 40 pivots, 15 iterations, and seed 0. All vertices and edges are rendered; panels preserve aspect ratio but use independent scales. Color encodes normalized BFS distance from the lowest-ID vertex. Geographic coordinates are not used.

**Degree heterogeneity and graph size.** We generated six Barabási–Albert preferential-attachment graphs using NetworkX: 2,000, 4,000, and 8,000 vertices, five attachments per new vertex, and graph seeds 0 and 1 at each size. The edge counts are 9,975, 19,975, and 39,975, respectively; maximum degrees range from 143 to 311. For each graph, we use five matched initial layouts (seeds 0–4), exact full stress, and the same 15-iteration schedule. We measure CPU, unsplit RoundRobin Full SGD, and the 16-segment schedule sequentially, alternating method order across initialization seeds after a preliminary run for each setting. Both GPU schedules wait for completion at each iteration, isolating segmentation from the earlier change in CPU wait placement. Across these six graphs, median CPU speedups range from 2.96 to 22.95× for 16 segments. Unsplit final stress ratios range from 1.170 to 1.519, compared with 1.007 to 1.038 after segmentation (Figure 7). Of the 30 matched layouts, 0 unsplit runs and 30 split runs finish within 5% of CPU. The split schedule takes 1.16–1.42 times the unsplit iteration time in graph-level medians. These measurements show the time–quality effect of interleaving within this graph family; they do not isolate degree distribution from other structural properties or establish behavior on all networks.

### 4.3 Scaling to Road Networks with Millions of Vertices

Beyond the CPU comparisons, we run RoundRobin Sparse SGD on `italy_osm` and `germany_osm`, with 6,686,493 and 11,548,845 vertices and 7,013,978 and 12,369,181 undirected edges, respec-

Table 1: Observed attainment within 15 iterations. Each row has ten paired seeds. Hit and End count RoundRobin runs reaching the target at any checkpoint and at iteration 15, respectively. Times are median cumulative SGD seconds at first observed attainment. CPU reached the target on all seeds. The RoundRobin time uses successful seeds only.

| Graph | Hit | End | CPU (s) | RR (s) |
|---|---|---|---|---|
| Full | | | | |
| USpowerGrid | 10 | 10 | 2.046 | 0.199 |
| dwt_2680 | 6 | 6 | 0.807 | 0.160 |
| bcsstk16 | 10 | 10 | 2.907 | 0.285 |
| fe_4elt2 | 10 | 10 | 11.593 | 0.463 |
| EVA | 9 | 9 | 1.678 | 0.281 |
| wiki-Vote | 10 | 10 | 6.691 | 0.385 |
| Sparse | | | | |
| luxembourg_osm | 10 | 10 | 6.722 | 0.149 |
| kron_g500-logn18 | 10 | 10 | 2.138 | 0.919 |
| coAuthorsCiteseer | 10 | 10 | 13.925 | 0.391 |

Table 2: Road-network runtime in seconds (one run per graph). SGD is the total for 15 iterations; algorithm time also includes preprocessing, setup, transfers, and postprocessing, but excludes input loading, stress evaluation, saving, and rendering.

| Graph | SGD (s) | Algorithm (s) |
|---|---|---|
| italy_osm | 2.11 | 43.45 |
| germany_osm | 4.48 | 80.32 |

tively. Both runs use 40 pivots, 15 iterations, $\varepsilon = 0.1$, and seed 0 on the same GPU, without the comparison experiments' stopping thresholds. We use the same reduced count for both networks to compare their runtimes under one memory budget. For Germany, storing one 32-bit distance for each vertex and pivot requires 1.72 GiB at 40 pivots, 8.60 GiB at 200, and 44.06 GiB at 1024. The last figure exceeds the laptop's 24 GiB of unified memory even before coordinates and other buffers. There are no paired CPU or Atomic runs at these scales.

Both layouts complete (Figure 8; Table 2). Germany takes 4.48 s for 15 iterations and 80.32 s including preprocessing and other algorithm work, demonstrating a completed layout beyond ten million vertices on one GPU. The largest verified input is not a measured capacity limit, and these 40-pivot runs do not establish equivalence to 200-pivot quality or CPU speedups. Preprocessing and data handling dominate overall time and remain important optimization targets.

The distinction between SGD time and algorithm time is especially important at this scale. SGD accounts for approximately 4.9% of algorithm time for Italy and 5.6% for Germany. Thus, accelerating the pair-update kernels further would address only a small part of the current time to produce coordinates. Common preprocessing alone took 37.50 s for Italy and 68.51 s for Germany. Both runs also evaluated full stress separately using 64 sampled sources, yielding $8.93 \times 10^{11}$ and $1.46 \times 10^{12}$, respectively. These absolute values describe different graphs and do not quantify the quality cost of reducing the pivot count. The run records contain no peak-memory measurement for either layout.

The pivot count provides a separate control over the size of the sparse problem. With fixed $p$, the $O(m+np)$ term count grows linearly with the numbers of edges and vertices. More pivots would represent long-range distances from more reference vertices. A larger-pivot experiment on a graph that fits in memory could test whether this improves full stress enough to justify its cost. Reducing $p$ from 200 to 40 reduces the vertex–pivot term budget by a factor of five, while leaving the actual-edge terms in place. It changes the sparse objective as well as storage and work, so the large-road experiment demonstrates feasibility under a stated configuration rather than extrapolating the CPU comparison results. The two completed runs establish a concrete operating point beyond ten million vertices; determining a maximum graph size would require a systematic sweep over graph structure, pivot count, and memory use.

## 5 Conclusion

Separating one-sided pivot updates from two-sided updates let RoundRobin Sparse SGD shorten iterations on all 37 comparable graphs while keeping median final full stress within 5% above the sequential CPU baseline on 36. The median algorithm speedup was $3.77\times$ once preprocessing and setup were included. At 40 pivots the method also completed a layout of 11.55 million vertices in 80.32 s on one Apple M4 Pro.

For Full SGD, interleaving 16 segments of each inner schedule across outer rounds gives shorter iterations on 17 of 19 comparable graphs and a median stress ratio at most 1.05 on 18. 55 of 60 diagnostic runs reach the same-seed quality target within 15 iterations. On wiki-Vote, median time to attainment is 0.385 s, compared with 6.691 s for CPU. The additional preferential-attachment study confirms a quality improvement over the unsplit schedule at an iteration-time cost. Two small graphs remain slower than CPU, and bcsstk17 retains a quality gap. Time to an application-specific stress threshold therefore remains a necessary complement to iteration time.

A GPU with more parallel execution units could process more independent pairs concurrently when a round contains enough work to occupy them. Dependencies between successive rounds still impose synchronization, and faster kernels leave preprocessing costs in place. Testing this prediction requires measurements on additional GPUs and a same-device comparison with destination-grouped updates [47].

## References


[1] R. Ahmed, F. De Luca, S. Devkota, S. Kobourov, and M. Li. Multicriteria Scalable Graph Drawing via Stochastic Gradient Descent, (SGD)$^2$. *IEEE Transactions on Visualization and Computer Graphics*, 28(6):2388–2399, 2022. doi: 10.1109/TVCG.2022.3155564 2

[2] J. Barnes and P. Hut. A hierarchical O(N log N) force-calculation algorithm. *Nature*, 324(6096):446–449, 1986. doi: 10.1038/324446a0 2

[3] K. Börsig, U. Brandes, and B. Pasztor. Stochastic Gradient Descent Works Really Well for Stress Minimization. In *Graph Drawing and Network Visualization (GD 2020)*, pp. 18–25, 2020. doi: 10.1007/978-3-030-68766-3_2 1, 2

[4] L. Bottou. Stochastic Gradient Descent Tricks. In *Neural Networks: Tricks of the Trade, 2nd ed.*, pp. 421–436. Springer, 2012. doi: 10.1007/978-3-642-35289-8_25 2

[5] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization Methods for Large-Scale Machine Learning. *SIAM Review*, 60(2):223–311, 2018. doi: 10.1137/16M1080173 2, 3

[6] U. Brandes and C. Pich. Eigensolver Methods for Progressive Multidimensional Scaling of Large Data. In *Graph Drawing (GD 2006), LNCS 4372*, pp. 42–53, 2007. doi: 10.1007/978-3-540-70904-6_6 2

[7] R. Davidson and D. Harel. Drawing graphs nicely using simulated annealing. *ACM Transactions on Graphics*, 15(4):301–331, 1996. doi: 10.1145/234535.234538 2

[8] T. A. Davis and Y. Hu. The University of Florida Sparse Matrix Collection. *ACM Transactions on Mathematical Software*, 38(1), 2011. Article 1, 25 pages. doi: 10.1145/2049662.2049663 2, 5

[9] J. de Leeuw. Convergence of the Majorization Method for Multidimensional Scaling. *Journal of Classification*, 5(2):163–180, 1988. doi: 10.1007/BF01897162 2

[10] O. Dekel, R. Gilad-Bachrach, O. Shamir, and L. Xiao. Optimal distributed online prediction using mini-batches. *Journal of Machine Learning Research*, 13:165–202, 2012. 2

[11] S. Di Bartolomeo, T. Crnovrsanin, D. Saffo, E. Puerta, C. Wilson, and C. Dunne. Evaluating Graph Layout Algorithms: A Systematic Review of Methods and Best Practices. *Computer Graphics Forum*, 43(6):e15073, 2024. doi: 10.1111/cgf.15073 2, 5

[12] T. Dwyer. Scalable, Versatile and Simple Constrained Graph Layout. *Computer Graphics Forum*, 28(3):991–998, 2009. doi: 10.1111/j.1467-8659.2009.01449.x 2

[13] T. Dwyer, Y. Koren, and K. Marriott. IPSep-CoLa: An Incremental Procedure for Separation Constraint Layout of Graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. doi: 10.1109/tvcg.2006.156 2

[14] T. Dwyer, K. Marriott, and P. J. Stuckey. Fast Node Overlap Removal. In *Graph Drawing*, vol. 3843 of *Lecture Notes in Computer Science*, pp. 153–164, 2006. doi: 10.1007/11618058_15 2

[15] P. Eades. A heuristic for graph drawing. *Congressus Numerantium*, 42:149–160, 1984. 2

[16] Y. Frishman and A. Tal. Multi-Level Graph Layout on the GPU. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1310–1317, 2007. doi: 10.1109/TVCG.2007.70580 2

[17] T. M. J. Fruchterman and E. M. Reingold. Graph Drawing by Force-Directed Placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991. doi: 10.1002/spe.4380211102 2

[18] P. Gajer, M. T. Goodrich, and S. G. Kobourov. A Multi-dimensional Approach to Force-Directed Layouts of Large Graphs. In *Graph Drawing*, vol. 1984 of *Lecture Notes in Computer Science*, pp. 211–221, 2001. doi: 10.1007/3-540-44541-2_20 2

[19] E. R. Gansner, Y. Hu, and S. North. A Maxent-Stress Model for Graph Layout. *IEEE Transactions on Visualization and Computer Graphics*, 19(6):927–940, 2013. doi: 10.1109/TVCG.2012.299 2

[20] E. R. Gansner, Y. Koren, and S. North. Graph Drawing by Stress Majorization. In *Graph Drawing (GD 2004), LNCS 3383*, pp. 239–250, 2005. doi: 10.1007/978-3-540-31843-9_25 1, 2

[21] gfx-rs developers. wgpu: portable graphics library for Rust. Project documentation. Accessed October 30, 2026. 5

[22] J. C. GOWER. Some distance properties of latent root and vector methods used in multivariate analysis. *Biometrika*, 53(3-4):325–338, 1966. doi: 10.1093/biomet/53.3-4.325 2

[23] F. Grötschla, J. Mathys, R. Veres, and R. Wattenhofer. CoRe-GD: A hierarchical framework for scalable graph visualization with GNNs. In *International Conference on Learning Representations*, 2024. 2

[24] M. Gürbüzbalaban, A. Ozdaglar, and P. A. Parrilo. Why Random Reshuffling Beats Stochastic Gradient Descent. *Mathematical Programming*, 186:49–84, 2021. doi: 10.1007/s10107-019-01440-w 2

[25] S. Hachul and M. Jünger. Drawing Large Graphs with a Potential-Field-Based Multilevel Algorithm. In *Graph Drawing*, vol. 3383 of *Lecture Notes in Computer Science*, pp. 285–295, 2005. doi: 10.1007/978-3-540-31843-9_29 2

[26] D. Harel and Y. Koren. A Fast Multi-Scale Method for Drawing Large Graphs. *Journal of Graph Algorithms and Applications*, 6(3):179–202, 2002. doi: 10.7155/jgaa.00051 2

[27] D. Harel and Y. Koren. Graph Drawing by High-Dimensional Embedding. *Journal of Graph Algorithms and Applications*, 8(2):195–214, 2004. doi: 10.7155/jgaa.00089 2

[28] Y. Hu. Efficient, High-Quality Force-Directed Graph Drawing. *The Mathematica Journal*, 10(1):37–71, 2005. 2

[29] M. Jacomy, T. Venturini, S. Heymann, and M. Bastian. ForceAtlas2, a Continuous Graph Layout Algorithm for Handy Network Visualization Designed for the Gephi Software. *PLoS ONE*, 9(6):e98679, 2014. doi: 10.1371/journal.pone.0098679 2

[30] R. Kaleem, S. Pai, and K. Pingali. Stochastic Gradient Descent on GPUs. In *Proceedings of the 8th Workshop on General Purpose Processing using GPUs (GPGPU-8)*, pp. 81–89, 2015. doi: 10.1145/2716282.2716289 2, 3

[31] T. Kamada and S. Kawai. An Algorithm for Drawing General Undirected Graphs. *Information Processing Letters*, 31(1):7–15, 1989. doi: 10.1016/0020-0190(89)90102-6 1, 2

[32] M. Khoury, Y. Hu, S. Krishnan, and C. Scheidegger. Drawing Large Graphs by Low-Rank Stress Majorization. *Computer Graphics Forum*, 31(3pt1):975–984, 2012. doi: 10.1111/j.1467-8659.2012.03090.x 2

[33] J. F. Kruiger, P. E. Rauber, R. M. Martins, A. Kerren, S. Kobourov, and A. C. Telea. Graph Layouts by t-SNE. *Computer Graphics Forum*, 36(3):283–294, 2017. doi: 10.1111/cgf.13187 2

[34] J. B. Kruskal. Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis. *Psychometrika*, 29(1):1–27, 1964. doi: 10.1007/BF02289565 2

[35] J. Li, J.-N. Schmelzle, Y. Du, S. Heumos, A. Guarracino, G. Guidi, P. Prins, E. Garrison, and Z. Zhang. Rapid GPU-Based Pangenome Graph Layout. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1–19, 2024. doi: 10.1109/sc41406.2024.00035 2

[36] M. Ortmann, M. Klimenta, and U. Brandes. A Sparse Stress Model. *Journal of Graph Algorithms and Applications*, 21(5):791–821, 2017. doi: 10.7155/jgaa.00440 1, 2, 3

[37] X. Pan, M. Lam, S. Tu, D. Papailiopoulos, C. Zhang, M. I. Jordan, K. Ramchandran, and C. Ré. Cyclades: Conflict-Free Asynchronous Machine Learning. In *Advances in Neural Information Processing Systems*, vol. 29, 2016. 2

[38] N. Pezzotti, J. Thijssen, A. Mordvintsev, T. Höllt, B. van Lew, B. P. F. Lelieveldt, E. Eisemann, and A. Vilanova. GPGPU Linear Complexity t-SNE Optimization. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1172–1181, 2020. doi: 10.1109/TVCG.2019.2934307 2

[39] B. Recht, C. Ré, S. Wright, and F. Niu. Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems*, vol. 24, 2011. 2

[40] H. Robbins and S. Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586 2

[41] R. A. Rossi and N. K. Ahmed. The Network Data Repository with Interactive Graph Analytics and Visualization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29, pp. 4292–4293, 2015. doi: 10.1609/aaai.v29i1.9277 2

[42] J. Sammon. A Nonlinear Mapping for Data Structure Analysis. *IEEE Transactions on Computers*, C-18(5):401–409, 1969. doi: 10.1109/t-c.1969.222678 2

[43] W. S. Torgerson. Multidimensional Scaling: I. Theory and Method. *Psychometrika*, 17(4):401–419, 1952. doi: 10.1007/bf02288916 2

[44] L. van der Maaten and G. Hinton. Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. 2

[45] C. Walshaw. A Multilevel Algorithm for Force-Directed Graph-Drawing. *Journal of Graph Algorithms and Applications*, 7(3):253–285, 2003. doi: 10.7155/jgaa.00070 2

[46] Y. Wang, Y. Wang, Y. Sun, L. Zhu, K. Lu, C. Fu, M. Sedlmair, O. Deussen, and B. Chen. Revisiting Stress Majorization as a Unified Framework for Interactive Constrained Graph Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):489–499, 2018. doi: 10.1109/TVCG.2017.2745919 2

[47] Z. Wang, M. Xue, Y. Wang, F. Zhong, and Y. Wang. Graph Drawing by Conflict-Free Parallel Stochastic Gradient Descent. *Journal of Computer-Aided Design & Computer Graphics*, 37(6):1063–1072, 2025. doi: 10.3724/SP.J.1089.2023-00627 2, 8

[48] J. Yu, Y. Hu, and X. Yuan. UNICON: A UNIform CONstraint Based Graph Layout Framework. In *Proceedings of the IEEE Pacific Visualization Symposium (PacificVis)*, pp. 61–70, 2022. doi: 10.1109/PacificVis53943.2022.00015 2

[49] J. X. Zheng, S. Pawar, and D. F. M. Goodman. Graph Drawing by Stochastic Gradient Descent. *IEEE Transactions on Visualization and Computer Graphics*, 25(9):2738–2748, 2019. doi: 10.1109/TVCG.2018.2859997 1, 2, 3, 5

[50] M. Zhu, W. Chen, Y. Hu, Y. Hou, L. Liu, and K. Zhang. DRGraph: An Efficient Graph Layout Algorithm for Large-scale Graphs by Dimensionality Reduction. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1666–1676, 2021. doi: 10.1109/TVCG.2020.3030447 2

[51] Y. Zhuang, W.-S. Chin, Y.-C. Juan, and C.-J. Lin. A Fast Parallel SGD for Matrix Factorization in Shared Memory Systems. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, pp. 249–256, 2013. doi: 10.1145/2507157.2507164 2